

Researcher: GitHub Repo Researcher (WF-GITHUB-RESEARCH)

	search/	<- Hybrid search: BM25 + vector, Reciprocal Rank
Fusion		
	embeddings/	<- Snowflake arctic-embed-xs (384D)
	group/	<- Cross-repo group management
	wiki/	<- LLM-powered wiki generation
	mcp/	<- MCP server: tools.ts, resources.ts, local-
backend.ts		
	cli/	
	gitnexus-web/	<- Vite + React browser UI (graph explorer + AI chat)
	gitnexus-shared/	<- Shared TypeScript types + constants
	gitnexus-clause-plugin/	<- Claude Code plugin metadata
	gitnexus-cursor-integration/	<- Cursor-specific integration
	.claude/skills/	<- Agent skills tự động cài khi analyze
	gitnexus-exploring/	
	gitnexus-debugging/	
	gitnexus-impact-analysis/	
	gitnexus-plan/	<- /gitnexus-plan slash command
	gitnexus-work/	<- /gitnexus-work (execute plan as atomic commits)
	gitnexus-review/	<- /gitnexus-review (PR review backed by graph)
	gitnexus-lfg/	<- /gitnexus-lfg (plan → gate → work → review pipeline)
	eval/	<- Evaluation harnesses
	pr-swarm-review/	<- Multi-persona PR review system

Công nghệ stack

Layer	Technology
---	---
Language	TypeScript (Node.js)
AST Parser	Tree-sitter (native bindings + WASM cho browser)
Graph DB	LadybugDB (custom graph DB tích hợp, tương tự LanceDB/Kuzu)
Search	BM25 + Snowflake arctic-embed-xs (384D vector), Reciprocal Rank Fusion
Community detection	Leiden algorithm
Protocol	MCP (Model Context Protocol — Anthropic standard)
Web UI	Vite + React
Packaging	npm (gitnexus package)
CI	GitHub Actions (composite actions)

3. Phân tích kỹ thuật

3.1 Ingestion Pipeline — 19-phase DAG

Pipeline chạy 19 phase theo DAG (Directed Acyclic Graph) — mỗi phase khai báo `deps` tường minh, runner dùng Kahn's topological sort để validate và thực thi. Không có plugin system — graph phase là compile-time static (`runner.ts`).

Thứ tự DAG:

```
scan → structure → [springConfig, markdown, cobol] → parse → [routes, tools, orm]
→ crossFile → scopeResolution → [springAutoConfiguration, springAop]
→ pruneLocalSymbols → mro → springAopInheritance → di → communities → processes
```

Mỗi phase nhận `PipelineContext` (shared mutable `KnowledgeGraph`) và `ReadonlyMap<string, PhaseResult>` chỉ chứa các dep đã khai báo — runner filter để ngăn hidden coupling. Pattern này đảm bảo phases không thể "đọc lên" output của phase không được khai báo là dep.

Sau pipeline, graph được load vào LadybugDB qua CSV streaming để không bị OOM với repo lớn.

3.2 Language-Agnostic Architecture — Plugin Pattern

Một trong những điểm thiết kế nổi bật nhất: toàn bộ logic xử lý ngôn ngữ được tách biệt hoàn toàn qua 2 interface:

LanguageProvider — cung cấp Tree-sitter queries, import semantics, MRO strategy:

```
// gitnexus/src/core/ingestion/language-provider.ts
interface LanguageProvider {
  id: string;
  extensions: string[];
  treeSitterQueries: string; // S-expressions cho Tree-sitter
  importSemantics: 'named' | 'wildcard-leaf' | 'wildcard-transitive' | 'namespace';
  mroStrategy: 'first-wins' | 'c3' | 'ruby-mixin' | 'none';
  // ...
}
```

ScopeResolver — xử lý call/import resolution, MRO, inheritance:

```
// gitnexus/src/core/ingestion/scope-resolution/contract/scope-resolver.ts
interface ScopeResolver {
  languageProvider: LanguageProvider;
  populateOwners(parsed: ParsedFile): void;
  buildMro(graph, parsed, nodeLookup): Map<DefId, DefId[]>;
  resolveImportTarget(target, fromFile, allFiles): string | null;
  // ... nhiều hook tùy chọn khác
}
```

Thêm ngôn ngữ mới = implement 2 interface + đăng ký vào `SCOPE_RESOLVERS` map. Không cần sửa shared code. CI auto-discover qua TypeScript `satisfies` constraint — thiếu language là compile error.

Unified capture tags (`@definition.class`, `@definition.function`, `@call.name...`) đảm bảo downstream extraction không cần branch theo ngôn ngữ.

3.3 Precomputed Relational Intelligence — Điểm cốt lõi

Khác với Traditional Graph RAG yêu cầu LLM tự query nhiều lần, GitNexus precompute tất cả tại thời điểm index:

- **Communities** (phase `communities`): Leiden algorithm phát hiện nhóm symbol liên quan (functional communities). Kết quả: agent có thể hỏi "module này làm gì?" thay vì đọc hàng chục file.
- **Processes** (phase `processes`): Trace execution flows từ entry points qua call chains. Kết quả: agent biết "LoginFlow đi qua 7 bước, bước 2 là `validateUser`" — một query thay vì traverse thủ công.
- **Impact** (MCP tool `impact`): Blast radius với depth grouping và confidence score. Kết quả: d=1 là WILL BREAK, d=2 là LIKELY AFFECTED — không cần agent tự suy luận.

3.4 MCP Tools — 17 công cụ

MCP server expose 17 tools qua giao thức chuẩn (không phải REST API riêng):

Tool	Chức năng quan trọng
---	---
<code>impact</code>	Blast radius — "cái gì phụ thuộc vào X?" với depth và

	confidence
context	360° view của một symbol — callers, callees, processes
query	Hybrid search BM25+vector, kết quả group theo process
detect_changes	Map git diff → affected symbols và processes (pre-commit check)
trace	Shortest path giữa 2 symbol (call + class-member edges)
rename	Multi-file rename với graph + text search, có dry_run
cypher	Raw Cypher query cho power users
route_map	API route → handler → consumer mappings
shape_check	Validate API response shape vs consumer property access
explain / pdg_query	Control/data flow analysis (cần --pdg index)

3.5 Agent Skills — Tự động cài khi analyze

gitnexus analyze tự động cài các skill files vào .claude/skills/ và .agents/skills/:

- **Standard skills:** exploring, debugging, impact-analysis, refactoring, guide, cli
- **Advanced skills:** gitnexus-plan, gitnexus-work, gitnexus-review, gitnexus-lfg
- **Repo-specific skills** (--skills flag): detect functional areas via Leiden community, generate per-area skill file mô tả key files, entry points, cross-area connections

Skill files là Markdown với description frontmatter để agent biết khi nào gọi (pattern giống với KZTEK .claude/commands/).

3.6 Worker Pool và Incremental Indexing

- Parse được thực hiện qua worker pool song song (không có sequential fallback, --workers 0 bị reject)
- Files chia thành chunk ~20MB để bound memory
- ParsedFile được serialize qua disk-backed store (không giữ trong RAM) — hỗ trợ repo cực lớn như Linux kernel
- Incremental: analyze detect stale bằng lastCommit == HEAD, skip nếu không đổi; detect_changes dùng git diff để re-index phần thay đổi
- Parse cache keyed theo file content hash + schema version — warm run không spawn worker

3.7 Điểm mạnh / Điểm yếu

Điểm mạnh:

- Precomputed intelligence: agent không cần nhiều turn để gather context
- Language-agnostic plugin system: thêm ngôn ngữ không đụng shared code
- MCP standard: tích hợp được với mọi AI agent hỗ trợ MCP
- Worker pool + disk-backed store: scale tới repo cực lớn
- Incremental indexing: analyze nhanh trên daily use
- Agent skills auto-install: zero-friction adoption
- Optional CFG/PDG (--pdg): control/data flow, taint analysis cho security

Điểm yếu:

- License PolyForm Noncommercial: **KHÔNG được dùng cho commercial use** — đây là hạn chế nghiêm trọng nếu tính tích hợp vào sản phẩm thương mại của KZTEK
- CFG/PDG chỉ có TypeScript/JavaScript — C#, Python... chưa có
- Node.js only: không phải language runtime phổ biến nhất trong stack KZTEK (C#/.NET)
- LadybugDB là custom DB — không có community support ngoài project này
- `npm install` nặng, yêu cầu C++ toolchain cho một số grammars (Dart, Proto, Swift, Kotlin)
- Web UI giới hạn ~5k files nếu dùng browser-only mode

4. Hiện trạng KZTEK

Khu vực tương ứng: Cơ chế cung cấp context codebase cho AI agents trong workflow KZTEK.

4.1 Hiện trạng thực tế (đã đọc trực tiếp)

KZTEK hiện dùng `code-graph/CODE-GRAPH.md` — file Markdown tĩnh, cập nhật thủ công bởi coding agents sau mỗi thay đổi code:

Trích `code-graph/CODE-GRAPH.md` (line 1–8):

"File này được duy trì tự động bởi coding agents."

"Đọc file này **TRƯỚC** khi đọc source code để hiểu cấu trúc dự án mà không cần mở từng file."

"**LƯU Ý QUAN TRỌNG:** Đây là AI Agent Framework workspace, **KHÔNG** phải codebase sản phẩm."

Workspace hiện tại là AI agent config framework — chưa có sản phẩm code thực tế. CODE-GRAPH.md ghi nhận cấu trúc `.claude/`, `KztekComponent/`, `scripts/` nhưng không có graph thực (không có call graph, community, process trace).

4.2 Tooling hiện có

Tool	Trạng thái	Ghi chú
---	---	---
<code>code-graph/CODE-GRAPH.md</code>	Tồn tại, thủ công	Cập nhật bởi agents sau code change, không tự động
<code>graphify</code> (PyPI <code>graphifyy</code>)	Tùy chọn, chưa cài	CLI Python, không có MCP exposure, không có community detection. GOTCHAS G006: tên package là <code>graphifyy</code> (2 chữ y)
MCP server codebase	Chưa có	Không có MCP server nào expose graph codebase
Automated AST indexing	Chưa có	Không có
Community/process tracing	Chưa có	Không có

4.3 Cơ chế hiện tại cho coding agents

Khi agent muốn hiểu codebase, theo CLAUDE.md §17.1:

1. Glob `code-graph/CODE-GRAPH.md` → Read file
2. Nếu trả lời được $\geq 3/5$ câu hỏi → bắt đầu coding
3. Thiếu info → Read thêm source file liên quan trực tiếp

Không có query API, không có blast-radius tool, không có process trace — agent phải Grep/Read thủ công.

5. So sánh GitNexus vs Hiện trạng KZTEK

Khía cạnh	GitNexus	KZTEK hiện tại
---	---	---
Indexing	Tự động, AST-based Tree-sitter	Thủ công bởi coding agent sau mỗi code change
Storage	LadybugDB graph DB (persistent, file <code>.gitnexus/</code>)	Markdown file <code>code-graph/CODE-GRAPH.md</code>
Query	17 MCP tools — <code>impact</code> , <code>context</code> , <code>query</code> , <code>trace</code> ...	Grep/Read thủ công; CODE-GRAPH giải đáp top-level câu hỏi
Community detection	Leiden algorithm tự động phát hiện functional cluster	Không có
Process tracing	Tự động trace execution flows từ entry points	Không có
Impact analysis	<code>impact</code> tool: blast radius 1 query, có confidence score	Phải đọc/trace thủ công — không có tool
Languages	14 ngôn ngữ (C#, TS, Python, Java, Go, Rust...)	N/A (workspace này là agent config, không phải product code)
Incremental update	git-diff based, chỉ re-parse phần thay đổi	Không có — cập nhật toàn bộ file thủ công
Agent skills	Tự động cài khi <code>analyze</code> (exploring, debugging, impact...)	Thủ công tạo <code>.claude/commands/*.md</code>
Protocol	MCP chuẩn — tương thích mọi AI agent	Proprietary — chỉ Claude Code đọc CLAUDE.md/CODE-GRAPH.md
License	PolyForm Noncommercial — cấm dùng thương mại	N/A

Nhận xét tổng: GitNexus và CODE-GRAPH.md của KZTEK giải quyết cùng vấn đề (cung cấp context codebase cho AI agent) nhưng ở quy mô và độ automation khác nhau hoàn toàn. CODE-GRAPH.md là giải pháp lightweight, phù hợp cho workspace config hiện tại. GitNexus phù hợp cho product codebase lớn (hàng nghìn file) cần real-time blast radius và cross-file tracing.

6. Thông tin repo

Mục	Chi tiết
---	---
URL	https://github.com/abhigyanpatwari/GitNexus
License	PolyForm Noncommercial 1.0.0 — KHÔNG dùng cho mục đích thương mại
npm package	<code>gitnexus</code> (npm)
Độ trưởng thành	Cao — có trendshift badge, Discord server, CI/CD đầy đủ, changelog, migration guide
Hoạt động	Rất tích cực — AGENTS.md last updated 2026-07-16, changelog và migration docs được duy trì
Stack	TypeScript + Node.js
Đặc điểm nổi bật	Self-dogfooding: GitNexus sử dụng chính GitNexus để index codebase của mình, sinh skills và CLAUDE.md/AGENTS.md tự động

7. Bảng đề xuất cải tiến KZTEK (Mode A — Bước 3b)

Lưu ý license: GitNexus dùng PolyForm Noncommercial — mọi đề xuất dưới đây học từ ý tưởng/pattern, không copy code nguồn. Mỗi cải tiến phải được viết lại từ đầu theo quy ước KZTEK.

Tổng hợp đề xuất

#	Đề xuất	Hiện trạng KZTEK	Học từ đâu trong GitNexus	Lý do thay đổi	Áp dụng vào đâu trong KZTEK	Đạt được gì	Rủi ro / Effort
--	-----	-----	-----	-----	-----	-----	-----
-	-		-----	-----	---	-	----
G X - 1	Depth-grouped impact taxonomy trong PR checklist	§15.3: cột "CODE-GRAPH impact" là free-text "liệt kê module/node... hoặc Không có" — agent tự suy luận, không có cấu trúc depth	impact MCP tool (3.4): trả về blast radius theo depth — d=1 "WILL BREAK" (caller trực tiếp), d=2 "LIKELY AFFECTED" (caller của caller) kèm confidence	Hiện tại agent viết tự do → reviewer không biết "Không có" là thật hay vì agent bỏ sót; không có sự phân biệt caller trực tiếp vs gián tiếp → review nông	§15.3 CLAUDE.md (sửa hướng dẫn + format mẫu); CODE-GRAPH-template.md (thêm cột Callers/Used-by)	PR checklist impact có cấu trúc 2-depth: depth-1 liệt kê module WILL BREAK (có thể đếm), depth-2 liệt kê LIKELY AFFECTED; reviewer có thể scan nhanh rủi ro mà không cần tự trace CODE-GRAPH	Thấp — chỉ sửa text trong 2 file template; không đổi code
G X - 2	Field deps tường minh trong PLAN-STEP frontmatter + validation bởi task-	PLAN-STEP-template.md: frontmatter chỉ có step, plan, agent, status, completed_at — không có deps; "Phụ thuộc" nằm trong body text, task-planner không validate	Ingestion pipeline (3.1): mỗi phase khai báo deps: [phaseA, phaseB] trong config; runner dùng Kahn's sort để validate DAG và ngăn hidden coupling giữa	task-planner có thể giao bước 3.1 khi 2.3 chưa done vì không có machine-readab	PLAN-STEP-template.md (thêm field deps: vào frontmatter); task-planner.md (thêm 1 validation step: trước khi mark bước là "ready", kiểm tra tất cả deps trong frontmatter có status ✓ không)	task-planner tự phát hiện nếu bước được giao khi dep chưa done → BLOCK + báo rõ "Bước 2.3 chưa ✓, không	Thấp-trung — sửa 2 file template + 1 đoạn validation logic trong task-planner.md; không ảnh hưởng plan cũ

	planne r		phases	le deps — người dùng phải đọc bảng MASTE R thủ công để kiểm tra		thể bắt đầu 3.1" thay vì chạy sai thứ tự; giảm sai sót trong plan có nhánh song song	(backwar d compatibl e — field mới không bắt buộc)
G X - 3	Skill /det ect- impa ct — semi- autom ate phần CODE- GRAP H impact trong PR	§15.3: agent phải tự đọc CODE-GRAPH.md, đối chiếu với files đã thay đổi, suy luận module bị ảnh hưởng, rồi viết impact section — tất cả thủ công, dễ bỏ sót khi code change lớn	detect_cha nges tool (3.4): nhận git diff → map file thay đổi → tìm node trong graph → trả về affected symbols và processes liên quan; một query thay vì traverse thủ công nhiều bước	Agent viết impact section dựa trên "nhớ từ CODE- GRAPH " hay "Grep nhánh" → dễ sót modul e có indirec t relatio nship; PR có nhiều file thay đổi → effort cao, chất lượng thấp	Tạo mới .claude/comm ands/detect- impact.md (skill /detect-impact); skill sẽ: (1) chạy git diff --name- only HEAD, (2) Read CODE- GRAPH.md, (3) traverse quan hệ Callers/Used-by cho mỗi module thay đổi, (4) output template-filled depth-1/depth-2 impact cho paste vào PR checklist	Thời gian viết PR checklist giảm từ "đọc CODE- GRAPH + suy luận thủ công" → "chạy skill + paste output"; chất lượng tăng vì bước (3) là determin istic lookup, không phụ thuộc "agent có nhớ"	Trung — tạo file skill mới + EVAL theo §18.5; cần CODE- GRAPH.m d có cột Callers /Used- by (phụ thuộc GX- 1 được áp dụng trước)
G X - 4	Cột Last verifi ed trong CODE- GRAP H để flag stale CONFIRMED	CODE-GRAPH- template.md: cột Confidence có 3 label (CONFIRMED/INFER RED/UNCERTAIN) nhưng không có ngày xác nhận; entry CONFIRMED từ 3 tháng trước vẫn hiển thị như mới — không có signal nào báo cần	Incremental indexing (3.6): detect stale bằng lastCommit == HEAD — nếu commit HEAD khác lần index cuối → re-parse; không giữ stale data âm	CONFIRMED entry hôm nay có thể sai sau khi code đổi — agent đọc CODE-	CODE-GRAPH- template.md (thêm cột Last verified: YYYY- MM-DD vào bảng Dependencies + Module chính); §17.2 CLAUDE.md (thêm rule: khi coding agent đọc CODE- GRAPH, nếu entry có Last verified cũ	Agent không còn tin mù vào CONFIRMED stale — phát hiện drift CODE- GRAPH vs code thực sớm	Thấp — sửa template + 3 dòng rule trong CLAUDE. md; không có automatio n, chỉ là quy tắc agent

	entries	re-verify	thăm	GRAPH tin vào CONFIRMED nhưng thực tế module đó đã được refactor; §17.2 chỉ nói "UNCERTAIN → phải đọc source " nhưng không có cơ chế biến CONFIRMED thành UNCERTAIN khi code thay đổi	hơn 30 ngày VÀ file đó xuất hiện trong git log tuần qua → downgrade tạm sang UNCERTAIN, re-verify trước khi dùng)	hơn; giảm bug "agent làm theo CODE-GRAPH cũ, code thực tế đã đổi"	phải tuân theo
G X - 5	Project-context skill hints tự sinh khi Phase 0 Audit (học từ skills auto-install pattern)	§3.0 Pre-0b: đọc docs/LESSONS.md 5-10 entry gần nhất; không có cơ chế tự sinh "gợi ý skill/command nào liên quan nhất cho project này" dựa trên CODE-GRAPH	gitnexus analyze (3.5): tự detect functional areas từ Leiden community detection → sinh per-area skill file mô tả key files, entry points, cross-area connections; zero-friction adoption	Mỗi session mới, agent phải đọc lại CODE-GRAPH để biết project đang có gì → nếu project có 20+ module, không thể đọc hết trong Pre-0; không	task-planner.md (thêm 1 bước trong Pre-0b: sau khi đọc CODE-GRAPH.md, đọc bảng Module chính → lọc module liên quan đến task slug → ghi <u>workspace/CONTEXT-HINTS.md</u> liệt kê: module liên quan, skill/command nên biết, UNCERTAIN entries cần watch out); hướng dẫn Dispatcher đọc file này trước giao việc cho agent đầu tiên	Dispatcher/agent đầu tiên nhận được "bản đồ rút gọn" (3-5 dòng) relevant cho task hiện tại thay vì phải đọc toàn bộ CODE-GRAPH; giảm 2-4 tool calls overhead mỗi session mới khi	Thấp-trung — sửa task-planner.md (thêm 1 bước) + hướng dẫn trong Pre-0b CLAUDE.md; không có gì phức tạp về kỹ thuật

				có "shortcut" gọi ý ngay khi bắt đầu task		CODE-GRAPH lớn	
G X - 6	Structured Handoff Payload keys (readonly deployment pattern)	PLAN-STEP-template.md: "Handoff Log — bước sau cần biết" là free-text với 4 gợi ý (Đã làm, File đã đọc/đổi, Quyết định, Bước sau cần biết); Dispatcher truyền "nguyên văn" Handoff Log vào prompt kế tiếp; agent kế tiếp có thể đọc cả phần "Đã làm" dài và không liên quan	PipelineContext pattern (3.1): phase chỉ nhận ReadonlyMap<string, PhaseResult> của các deployment filter đã khai báo — để ngăn phase sau đọc lên output của phase không phải deployment; loại bỏ hidden coupling	Agent kế tiếp nhận Handoff Log dài (có cả "Đã làm" mô tả chi tiết) → phải đọc toàn bộ để tìm "bước sau cần biết"; mặt khác agent có thể bỏ qua do log dài → bỏ sót thông tin quan trọng	PLAN-STEP-template.md (chia "Handoff Log" thành 3 key rõ: do_not_redo, watch_out, next_inputs); CLAUDE.md §16.4 (hướng dẫn Dispatcher chỉ trích xuất 3 key này để nhúng vào prompt kế tiếp, không truyền toàn bộ section "Đã làm")	Prompt của agent kế tiếp giảm noise: chỉ nhận 3 key cần thiết thay vì toàn bộ log; giảm trường hợp "agent bỏ qua Handoff Log vì quá dài" — 3 key ngắn dễ đọc hơn đoạn văn tự do	Thấp — chỉ sửa template + 2 đoạn hướng dẫn trong CLAUDE.md §16.4; backward compatible với step files hiện có (chỉ áp dụng cho plan mới)

Chi tiết từng đề xuất

GX-1: Depth-grouped impact taxonomy trong PR checklist

Học từ: **impact** MCP tool trong GitNexus — trả về blast radius với depth grouping (d=1: caller trực tiếp → WILL BREAK; d=2: caller của caller → LIKELY AFFECTED) và confidence score dựa trên type-of-edge. File tham chiếu: `gitnexus/src/mcp/tools.ts` (tool **impact**).

Hiện trạng KZTEK: §15.3 CLAUDE.md ghi:

"CODE-GRAPH impact: [liệt kê module/node bị ảnh hưởng (thêm/xóa/rename module, thay đổi API, DB schema, dependency), hoặc "Không có"]"

Không có hướng dẫn depth, không có format mẫu → mỗi agent viết khác nhau. CODE-GRAPH-template.md không có cột **Callers/Used-by** → agent không có dữ liệu để traverse.

Thay đổi cần làm:

1. Trong `CODE-GRAPH-template.md` — bảng Module chính: thêm cột `Callers/Used-by` (danh sách module nào phụ thuộc vào module này)
2. Trong §15.3 CLAUDE.md — thay text hiện tại bằng:

```
- CODE-GRAPH impact:  
  - Depth-1 (WILL BREAK): [module/node CÓ quan hệ trực tiếp với phần đã thay đổi — liệt kê từ cột "Callers/Used-by" trong CODE-GRAPH]  
  - Depth-2 (LIKELY AFFECTED): [module/node gọi các module ở depth-1 — liệt kê nếu rõ ràng, hoặc "Cần trace thêm"]  
  - Nếu không có quan hệ: "Isolated change — không có module nào phụ thuộc"
```

Kết quả đạt được: Tech Lead nhìn vào PR checklist thấy ngay "depth-1: 2 module WILL BREAK" thay vì "module A liên quan" mơ hồ → quyết định review có depth ứng với rủi ro thực tế.

GX-2: Field `deps` tường minh trong PLAN-STEP frontmatter

Học từ: Ingestion pipeline DAG của GitNexus — mỗi phase object có `deps: string[]` tường minh; runner thực hiện Kahn's topological sort, phát hiện cycle trước khi chạy. File tham chiếu:

[gitnexus/src/core/ingestion/pipeline-phases/runner.ts](#).

Hiện trạng KZTEK: PLAN-STEP-template.md hiện tại chỉ có frontmatter:

```
---  
step: N.M  
plan: ../PLAN-MASTER.md  
agent: [agent phụ trách]  
status: todo  
completed_at:  
---
```

Không có `deps` — task-planner kiểm tra deps bằng đọc PLAN-MASTER thủ công, không machine-readable.

Thay đổi cần làm:

1. Trong PLAN-STEP-template.md: thêm `deps: []` vào frontmatter (optional, để trống = no deps)
2. Trong task-planner.md: trước khi giao bước N.M cho agent, chạy pre-check:

- Đọc frontmatter `deps` của step file
- Với mỗi dep step ID → tìm trong PLAN-MASTER, kiểm tra status ✓
- Nếu dep chưa done → BLOCK với thông báo "Bước [dep] chưa ✓"

Kết quả đạt được: Giảm từ "task-planner phải nhớ kiểm tra thủ công" → "validation tự động trước mỗi bước" → không bao giờ giao bước 3.1 khi 2.3 chưa xong, dù plan có nhánh song song phức tạp.

GX-3: Skill `/detect-impact`

Học từ: `detect_changes` MCP tool của GitNexus — nhận git diff → map sang affected graph nodes → trả về theo process/functional area. Không cần agent tự traverse. File tham chiếu: [gitnexus/src/mcp/tools.ts](#) (tool `detect_changes`).

Hiện trạng KZTEK: Không có skill tương đương. Agent viết PR checklist impact section theo trí nhớ / Grep thủ công.

Thay đổi cần làm: Tạo `.claude/commands/detect-impact.md` với flow:

1. `git diff --name-only HEAD` (lấy danh sách file thay đổi)
2. Read `code-graph/CODE-GRAPH.md` (bảng Module + cột Callers/Used-by)
3. Với mỗi file thay đổi → tìm module chứa nó → lấy Callers/Used-by → `depth-1 list`
4. Với mỗi `depth-1 module` → lấy Callers/Used-by của chúng → `depth-2 list`
5. Output: template-filled impact section sẵn để paste vào PR checklist §15.3

Tạo `.claude/evals/detect-impact.md` với 3 CE (theo EDD §18.5) trước khi tạo skill.

Lưu ý: Skill này phụ thuộc GX-1 (cột `Callers/Used-by` phải có trong `CODE-GRAPH-template.md` trước).

Nếu GX-1 chưa áp dụng → skill vẫn có thể hoạt động nhưng `depth-1/depth-2` sẽ thiếu chính xác.

Kết quả đạt được: Thời gian viết PR impact section giảm từ ~5-10 phút (đọc + trace thủ công) → <1 phút (chạy skill + paste); chất lượng nhất quán vì là lookup không phụ thuộc "agent có nhớ."

GX-4: Cột `Last verified` trong CODE-GRAPH

Học từ: Incremental indexing của GitNexus — detect stale bằng `lastCommit == HEAD`, re-index phần thay đổi, không giữ data cũ âm thầm. Confidence được tự động downgrade khi phát hiện stale. File tham chiếu: `gitnexus/src/core/ingestion/pipeline-phases/` (phase structure).

Hiện trạng KZTEK: Confidence labels (CONFIRMED/INFERRED/UNCERTAIN) theo §17.2 không có ngày xác nhận. Entry CONFIRMED từ ngày tạo CODE-GRAPH giữ nguyên label dù code đã đổi nhiều lần.

Thay đổi cần làm:

1. `CODE-GRAPH-template.md`: thêm cột `Last verified` (YYYY-MM-DD) vào bảng Dependencies và Module chính
2. §17.2 CLAUDE.md: thêm rule — khi coding agent đọc CODE-GRAPH trước task, nếu gặp entry có `Last verified > 30` ngày trước ngày hiện tại VÀ module đó xuất hiện trong output `git log --oneline -7` → agent PHẢI re-verify bằng cách đọc source file trực tiếp trước khi dùng thông tin đó, không tin vào label CONFIRMED cũ

Kết quả đạt được: Agent phát hiện được "module này CONFIRMED từ 45 ngày trước và đã có 3 commit sửa nó tuần qua → cần re-verify" — giảm bug do agent tin CODE-GRAPH lạc hậu. Số lần agent làm sai vì dữ liệu CODE-GRAPH stale có thể đo bằng số incident/bug report loại này trong `docs/LESSONS.md`.

GX-5: Project-context skill hints tự sinh (Pre-0 Audit)

Học từ: Agent skills auto-install của GitNexus — `gitnexus analyze` detect functional areas qua Leiden community, sinh per-area skill file mô tả key files và cross-area connections. File tham chiếu: `gitnexus/.claude/skills/` (generated skill files), `gitnexus/src/core/ingestion/pipeline-phases/communities.ts`.

Hiện trạng KZTEK: §3.0 Pre-0b — đọc `docs/LESSONS.md` 5-10 entry gần nhất. Không có bước nào gợi ý "skill/command nào liên quan" dựa trên module CODE-GRAPH của task hiện tại.

Thay đổi cần làm:

1. `task-planner.md`: sau Pre-0b, thêm bước lightweight:
 - Đọc `CODE-GRAPH.md` bảng Module chính
 - So sánh tên task slug với tên/mục đích module (string match đơn giản)
 - Tìm trong `.claude/commands/` các skill liên quan (Glob + đọc `description` frontmatter)

- Ghi `_workspace/CONTEXT-HINTS.md` (≤ 20 dòng): module liên quan, skill nên biết, UNCERTAIN entries cần watch

2. Dispatcher nhúng nội dung `CONTEXT-HINTS.md` vào đầu prompt của agent đầu tiên trong chain.

Kết quả đạt được: Agent đầu tiên trong chain nhận ngay "module liên quan: X, Y; skill nên dùng: /detect-impact; UNCERTAIN: Z" thay vì phải đọc toàn bộ CODE-GRAPH rồi suy luận — giảm 2-4 tool calls overhead mỗi session mới, đặc biệt quan trọng khi product codebase KZTEK phát triển lớn hơn.

GX-6: Structured Handoff Payload keys

Học từ: PipelineContext readonly dep isolation trong GitNexus — mỗi phase chỉ thấy `ReadonlyMap<string, PhaseResult>` của các dep đã khai báo, runner filter trước khi truyền → ngăn hidden coupling. File tham chiếu: `gitnexus/src/core/ingestion/pipeline-phases/runner.ts` (context building logic).

Hiện trạng KZTEK: §16.4 Handoff Log là free-text với 4 dấu gạch đầu dòng gợi ý:

- Đã làm: [tóm tắt 2-3 câu]
- File/module đã đọc hoặc đổi: [đường dẫn]
- Quyết định quan trọng: [nếu có]
- Bước sau cần biết: [cảnh báo / gotcha / điều KHÔNG cần làm lại]

Dispatcher truyền "nguyên văn nội dung" — agent kế tiếp phải đọc toàn bộ 4 mục, trong đó "Đã làm" và "File đã đọc" thường dài và không cần thiết cho bước kế tiếp.

Thay đổi cần làm:

1. PLAN-STEP-template.md: chia Handoff Log thành section rõ:

```
## Handoff Payload — bước sau đọc phần này (chỉ phần này, không cần đọc "Đã làm")
- do_not_redo: [thao tác đã làm xong, bước sau KHÔNG làm lại — vd: "đã clone, không cần clone lại"]
- watch_out: [gotcha / điều kiện bất ngờ bước sau cần biết — vd: "branch X đang ở dirty state"]
- next_inputs: [artifact/file/quyết định bước sau cần làm input — vd: "dùng commit hash abc1234"]
```

2. §16.4 CLAUDE.md: hướng dẫn Dispatcher chỉ trích 3 key trên để nhúng vào prompt kế tiếp — không truyền section "Đã làm" trừ khi bước kế tiếp đặc biệt cần biết chi tiết.

Kết quả đạt được: Prompt của agent kế tiếp giảm ~40-60% noise (bỏ phần "Đã làm" dài); 3 key rõ ràng để đọc hơn đoạn văn tự do → giảm trường hợp agent bỏ qua Handoff Log vì "quá dài, không biết phần nào quan trọng."